

Q. Write a program in C to find the Second Smallest element in an array.

-7

```
#include <stdio.h>
#include <limits.h>

int main()
{
    int n;
    printf("Enter the number of array elements to be inserted:");
    scanf("%d", &n);
    if (n < 2) {
        printf("Enter more than one element in the array:");
    }

    int arr[n];
    printf("Enter %d elements:", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int smallest = INT_MAX;
    int secondSmallest = INT_MAX;
    for (int i = 0; i < n; i++) {
        if (arr[i] < smallest) {
            secondSmallest = smallest;
            smallest = arr[i];
        }
        else if (arr[i] > smallest && arr[i] < secondSmallest) {
            secondSmallest = arr[i];
        }
    }

    if (secondSmallest == INT_MAX) {
        printf("There is no second smallest element");
    }
    else {
```

```
        printf("the SecondSmallest element : %d", SecondSmallest);  
    }  
    return 0;  
}
```

Output:

enter the number of array elements to be inserted : 5
enter 5 elements : 10 20 15 13 11
the Second Smallest element : 11

- b. Write a program to find the sum of the left diagonal of a matrix.

-7

```
#include <stdio.h>
int main()
{
    int n, i, j;
    int matrix[10][10];
    int sum = 0;
    printf("Enter the order of the matrix: ", n);
    scanf("%d", &n);
    printf("Enter matrix element: \n");
    for(i=0; i<n; i++)
    {
        for(j=0; j<n; j++)
        {
            scanf("%d", &matrix[i][j]);
        }
    }
    for(i=0; i<n; i++)
    {
        sum = sum + matrix[i][i];
    }
    printf("The sum of left diagonal matrix: %d", sum);
    return 0;
}
```

Output:

Enter the order of matrix: 2
 Enter matrix elements: 10 20 30 40
 The sum of left diagonal matrix: 50

1. Write a C program to simulate the following CPU Scheduling algorithm to find turnaround time and waiting time.

(a) FCFS

```
#include <stdio.h>
```

```
int main ()
```

```
{
```

```
int n, i, Completed=0, Current-time=0, Shortest;
```

```
printf("Enter number of processes:");
```

```
scanf("%d", &n);
```

```
int at[n], bt[n], ct[n], tat[n], wt[n], visited[n];
```

```
printf("Enter Arrival Time and Burst Time:\n");
```

```
for (i=0; i<n; i++) {
```

```
printf("P.d AT BT :", i+1);
```

```
scanf("%d %d", &at[i], &bt[i]);
```

```
visited[i]=0;
```

```
}
```

```
while (Completed < n) {
```

```
Shortest = -1;
```

```
for (i=0; i<n; i++) {
```

```
if (at[i] <= Current-time & visited[i]==0) {
```

```
if (Shortest == -1 || bt[i] < bt[Shortest]) {
```

```
Shortest = i;
```

```
}
```

```
}
```

```
}
```

```
#include <stdio.h>
```

```
int main()
```

```
int n, i;
```

```
printf("Enter number of processes:");
```

```
scanf("%d", &n);
```

```
int at[n], bt[n], wt[n], tat[n], rt[n], ct[n];
```

```
for (i=0; i<n; i++){
```

```
    printf("\n Process %d\n", i+1);
```

```
    printf("Arrival Time:");
```

```
    scanf("%d", &at[i]);
```

```
    printf("Burst Time:");
```

```
    scanf("%d", &bt[i]);
```

```
}
```

```
ct[0] = at[0] + bt[0];
```

```
tat[0] = ct[0] - at[0];
```

```
wt[0] = tat[0] - bt[0];
```

```
rt[0] = wt[0];
```

```
for (i=1; i<n; i++){
```

```
    if (ct[i-1] < at[i])
```

```
        ct[i] = at[i] + bt[i];
```

```
    else
```

```
        ct[i] = ct[i-1] + bt[i];
```

```
tat[i] = ct[i] - at[i];
```

```
wt[i] = tat[i] - bt[i];
```

```
rt[i] = wt[i];
```

proof ("Average Waiting Time = 1.2 f", total-wt/n);
proof ("Average Turnaround Time = 1.2 f", total-tat/n);
proof ("Average Response Time = 1.2 f/n", total-rt/n);
return 0;

3

Process : 1

Process: 2

Arrival Time : 0

Burst Time : 4

Rate

Procell : 4

Arrival Time: 0

Burst Time : 8

AT BT

6

2

2

6

9

6

○

4

6

2

6

2

0

6

12

6

15

9

0

8

20

12

9

r

Waiting Time: 5.00

Turnaround Time = 10.0

Response Time = 5.00

(b) SJF (Preemptive and Non-Preemptive)

-7

```
#include <stdio.h>
int main()
{
    int n;
    printf("Enter number of Processes:");
    scanf("%d", &n);
```

```
    int pid[n], at[n], bt[n];
    for (int i=0; i<n; i++)
    {
```

```
        pid[i] = i+1;
        printf("\n Process %d\n", pid[i]);
        printf("Arrival Time:");
        scanf("%d", &at[i]);
        printf("Burst Time:");
        scanf("%d", &bt[i]);
    }
```

// Non Preemptive SJF //

```
int ct[n], start[n], end[n], rt[n];
```

```
int Completed[n];
```

```
int time = 0, count = 0;
```

```
for (int i=0; i<n; i++)
    Completed[i] = 0;
```

```
while (count < n)
```

```
{
```

```
    int minIndex = -1;
```

```
    int minBT = 9999;
```

```
for (int i=0; i<n; i++)
```

```
{
    if (at[i] <= time && Completed[i]==0)
```

```
    {
        if (bt[i] < minBT)
```

```
        {
            minBT = bt[i];
```

```
            minIndex = i;
```

```
        }
```

```
    }
```

```
}
```

```
if (minIndex == -1)
```

```
{
    time++;
```

```
}
```

```
else {
```

```
    rt1[minIndex] = time - at[minIndex];
```

```
    time += bt[minIndex];
```

```
    ct1[minIndex] = time;
```

```
    tot1[minIndex] = ct1[minIndex] - at[minIndex];
```

```
    cut1[minIndex] = tot1[minIndex] - bt[minIndex];
```

```
    Completed[minIndex] = 1;
```

```
    Count++;
```

```
}
```

```
}
```

```
// Preemptive SJF (SRTF) //
```

```
int at2[n], tot2[n], cut2[n], rt2[n];
```

```
int rt = bt[0];
```

```
int finished = 0;
```

```
time = 0;
```

```
for (int i = 0; i < n; i++) {
    st_bt[i] = bt[i];
    st2[i] = -1;
}
```

```
while (finished < n) {
    int minIndex = -1;
    int minBT = 9999;
```

```
for (int i = 0; i < n; i++) {
    if (st[i] <= time && st_bt[i] > 0) {
        if (st_bt[i] < minBT) {
            min_BT = st_bt[i];
            minIndex = i;
        }
    }
}
```

```
if (minIndex == -1) {
    time++;
}
```

```
else {
```

```
    if (st2[minIndex] == -1)
```

```
        st2[minIndex] = time - st[minIndex];
```

```
    st_bt[minIndex]--;
```

```
    time++;
```

```
    if (st_bt[minIndex] == 0) {
```

```
        st2[minIndex] = time;
```

```
        st2[minIndex] = st2[minIndex] - st[minIndex];
```

```
        st2[minIndex] = st2[minIndex] - bt[minIndex];
```

```
        finished++;
```

```
    }
```

```
}
```

```
float avgWT=0, avgTAT=0, avgRT=0;
printf("\n\n ----- NON PREEMPTIVE SJF ----- \n\n");
printf("PID | P1 | P2 | P3 | P4 | P5 | P6 | P7 | P8 | P9 | P10 |");
```

```
for (int i=0; i<n; i++)
```

```
{
    printf("P%d | %d | %d | %d | %d | %d | %d | %d | %d | %d | %d |",
        pid[i], at[i], bt[i], ct[i], tat[i], wt[i], rt[i], ...);
```

```
    avgWT += wt[i];
```

```
    avgTAT += tat[i];
```

```
    avgRT += rt[i];
```

```
}
```

```
printf("\n Average WT = %.2f", avgWT/n);
```

```
printf("\n Average TAT = %.2f", avgTAT/n);
```

```
printf("\n Average RT = %.2f", avgRT/n);
```

```
avgWT = avgTAT = avgRT = 0;
```

```
printf("\n\n ----- PREEMPTIVE SJF (SRTF) ----- \n\n");
```

```
printf("PID | P1 | P2 | P3 | P4 | P5 | P6 | P7 | P8 | P9 | P10 |");
```

```
for (int i=0; i<n; i++)
```

```
{
    printf("P%d | %d | %d | %d | %d | %d | %d | %d | %d | %d | %d |",
```

```
        pid[i], at[i], bt[i], ct[i], tat[i], wt[i], rt[i], ...);
```

```
    avgWT += wt[i];
```

```
    avgTAT += tat[i];
```

```
    avgRT += rt[i];
```

```
}
```

```

printf ("In Average WT = %.2f", avgWT/n);
printf ("In Average TAT = %.2f", avgTAT/n);
printf ("In Average RT = %.2f", avgRT/n);

```

```

return 0;

```

```

}

```

Output

Enter number of processes : 4

Process 1

Arrival Time : 0

Burst Time : 8

Process 2

Arrival Time : 1

Burst Time : 4

Process 3

Arrival Time : 2

Burst Time : 9

Process 4

Arrival Time : 3

Burst Time : 3

	Non PREEMPTIVE SJF					
PID	AT	BT	CT	TAT	WT	RT
P1	0	8	8	8	0	0
P2	1	4	15	14	10	10
P3	2	9	24	22	13	13
P4	3	3	11	8	5	5

Average WT = 7.00

Average TAT = 13.00

Average RT = 7.00

----- PREEMPTIVE SJF (SRTF) -----

PID	AT	BT	CT	TAT	WT	RT
P1	0	8	15	15	7	0
P2	1	4	5	4	0	0
P3	2	9	24	22	13	13
P4	3	3	8	5	2	2

Average WT = 5.50

Average TAT = 11.50

Average RT = 3.75

(c) Priority (pre-emptive & Non-pre-emptive)

->

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    int p[n], at[n], bt[n], pr[n];
```

```
    for (int i=0; i<n; i++)
```

```
    {
```

```
        p[i] = i+1;
```

```
        printf("Process %d\n", p[i]);
```

```
        printf("Arrival Time: ");
```

```
        scanf("%d", &at[i]);
```

```
        printf("Burst Time: ");
```

```
        scanf("%d", &bt[i]);
```

```
        printf("Priority: ");
```

```
        scanf("%d", &pr[i]);
```

```
    }
```

```
    // Non-Preemptive Priority //
```

```
    int at[n], tot[n], out[n], rt[n], completed[n];
```

```
    int time = 0, count = 0;
```

```
    for (int i=0; i<n; i++)
```

```
        completed[i] = 0;
```

```
    printf("Non-Preemptive Priority Scheduling Chart ---\n")
```

```

while (count < 0)
{
    int minp = 999, index = -1;
    for (int i = 0; i < n; i++)
    {
        if (at[i] <= time && completed[i] == 0)
        {
            if (p[i] < minp)
            {
                minp = p[i];
                index = i;
            }
        }
    }

    if (index == -1)
        time++;
    else
    {
        printf("P%d\n", pid[index]);
        at[index] = time - at[index];
        time += bt[index];

        ct[index] = time;
        tot[index] = ct[index] - at[index];
        wt[index] = tot[index] - bt[index];

        completed[index] = 1;
        count++;
    }
}

```

```
for(int i=0; i<n; i++)
```

$$\text{total } W_T = \text{wt } [Z_i J];$$
$$\text{total } [AT+] = \text{tot } [i];$$
$$\text{total RT} = \sum \text{RT}_i$$

7

priority ("In Average Turnaround Time = $\frac{1}{n} \cdot d^n$, total TAT/h).

print ("In Average Response Time is %.2f s", totalRT/len);

11 Preemptive Priority 11

int $\text{net}[n]$, $\text{ct}[n]$, $\text{tat}[n]$, $\text{wt}[n]$, $\text{at}[n]$;

it finished = 0;

time $\rightarrow 0$;

for $i \geq 0$; $i \leq 0$, $i+1$

4

$$nb + [i] = bt[i];$$
$$x + 2xi = -1;$$

2

priority ("lnln --- Priority Preemptive Gantt chart --- |n|")

while finished so)

5

int rmp = 909, index = -1;

```
for(int i=0; i<n; i++)  
{  
    if (arr[i] <= time && nbt[i] > 0)  
    {  
        if (arr[i] < minp)  
        {  
            minp = arr[i];  
            index = i;  
        }  
    }  
    if (index == -1)  
        time++;  
    else  
    {  
        printf ("P.L.d %d", arr[index]);  
        if (arr[index] == -1)  
            arr[index] = time - arr[index];  
        nbt[index]--;  
        time++;  
    }  
    if (nbt[index] == 0)  
    {  
        arr[index] = time;  
        arr[index] = arr[index] - arr[index];  
        arr[index] = arr[index] - arr[index];  
        finished++;  
    }  
}
```


Round Robin

-7

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, tq;
```

```
    printf("Enter Number of Processes:");
```

```
    scanf("%d", &n);
```

```
    int at[n], bt[n], rt[n];
```

```
    int ct[n], tot[n], wt[n], response[n];
```

```
    int visited[n];
```

```
    printf("Enter Arrival Time and Burst Time:\n");
```

```
    for (int i=0; i<n; i++)
```

```
    {
```

```
        printf("P%d AT BT:", i+1);
```

```
        scanf("%d %d", &at[i], &bt[i]);
```

```
        rt[i] = bt[i];
```

```
        visited[i] = 0;
```

```
    }
```

```
    printf("Enter Time Quantum:");
```

```
    scanf("%d", &tq);
```

```
    int time = 0, Completed = 0;
```

```
    printf("\n Gantt Chart:\n");
```

```
    while (Completed < n)
```

```
    {
```

```
        int done = 1;
```

```

for (int i = 0; i < n; i++)
{
    if (rt[i] > 0 && at[i] <= time) {
        done = 0;

        if (visited[i] == 0) {
            response[i] = time - at[i];
            visited[i] = 1;
        }

        if (rt[i] > tq)
        {
            printf("%d P1.d", i+1);
            time += tq;
            rt[i] -= tq;
        }
        else {
            printf("%d P2.d", i+1);
            time += rt[i];
            at[i] = time;
            rt[i] = 0;
            completed++;
        }
    }
}

if (done)
    time++;
}

printf("\n");
float avg_at = 0, avg_wt = 0, avg_rt = 0;

printf("\n Process \t AT \t BT \t CT \t RT \t WT \t RT \n");

```

```
for(int i=0; i<n; i++)
```

$$tot[i] = tot[i] - at[i];$$
$$cut[i] = tot[i] - bt[i];$$
$$\text{avg_tot} += \text{tot}[i];$$

avg-cut + = cut[i];

avg = st + nelpade [i];

[illegible]

$i+1, at[i], bt[i], t[i], tat[i], ut[i], response[i]$

3

proof ("In Average TAT = 1.2f", avg-tat (n));

proof ("10 Average WT = 7.2g", avg-wt 10);

pred (10 average RT = 7.26", avg-st 1n);

motion 0;

3

Output:

Enter number of procells: 4

Enter Arrival Time and Depart Time:

P1 A1 B1: 09

P2 A7 B7: 15

P3 AT BI: 3 4

Q4 AT BT: 56

Enter Time Quarter : 2

Cost cost:

$|P_1| |P_3| |P_3| |P_4| |P_2| |P_3| |P_4| |P_2| |P_4|$

Process	AT	BT	CT	TAT	WT	RT
P1	0	2	2	2	0	0
P2	1	5	15	14	9	1
P3	3	4	12	9	5	1
P4	5	6	17	12	6	1

Average TAT = 9.25

Average WT = 5.00

Average RT = 0.75

Enter Time Quantum: 3

Gantt chart:

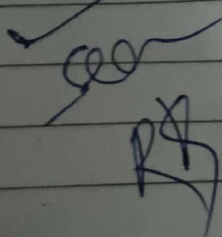
P1 | P2 | P3 | P4 | P2 | P3 | P4

Process	AT	BT	CT	TAT	WT	RT
P1	0	2	2	2	0	0
P2	1	5	13	12	7	1
P3	3	4	14	11	7	2
P4	5	6	17	12	6	3

Average TAT = 9.25

Average WT = 5.00

Average RT = 1.50



Write a C program to Simulate Multi-level queue Scheduling algorithm Considering the following Scenario. All the processes in the System are divided into two categories - System processes and user processes. System processes are to be given higher priority than user processes. Use FIFO Scheduling for the processes in each queue.

```
#include <stdio.h>
```

```
#define MAX 100
```

```
int main () {
```

```
    int n;
```

```
    int pid[MAX], arrival[MAX], burst[MAX], type[MAX];
```

```
    int ct[MAX], tat[MAX], wt[MAX], rt[MAX], Hrt[MAX];
```

```
    int sys_pid[MAX], sys_arr[MAX], sys_burst[MAX];
```

```
    int user_pid[MAX], user_arr[MAX], user_burst[MAX];
```

```
    int sys_index[MAX], user_index[MAX];
```

```
    int sys_n = 0, user_n = 0;
```

```
    printf("Enter Number of Processes:");
```

```
    scanf("%d", &n);
```

```
    for(int i=0; i<n; i++) {
```

```
        printf("\n Process %d\n", i+1);
```

```
        pid[i] = i+1;
```

```
printf("Enter Arrival Time:");
scanf("%d", &arrival[i]);
```

```
printf("Enter Burst Time:");
scanf("%d", &burst[i]);
```

```
printf("Enter Type (1 = System, 2 = user):");
scanf("%d", &type[i]);
```

```
}
```

```
for(int i=0; i<n; i++) {
    if(type[i]==1) {
        sys_pid[sys_n] = pid[i];
        sys_arr[sys_n] = arrival[i];
        sys_burst[sys_n] = burst[i];
        sys_index[sys_n] = i;
        sys_n++;
    }
```

```
    else {
```

```
        user_pid[user_n] = pid[i];
        user_arr[user_n] = arrival[i];
        user_burst[user_n] = burst[i];
        user_index[user_n] = i;
        user_n++;
    }
```

```
}
```

```
for(int i=0; i<sys_n-1; i++) {
    for(int j=i+1; j<sys_n; j++) {
        if(sys_arr[i] > sys_arr[j]) {
            int temp;
```

```

temp = sys-arm[i]; sys-arm[i] = sys-arm[j]; sys-arm[j] = temp;
temp = sys-burst[i]; sys-burst[i] = sys-burst[j]; sys-burst[j] = temp;
temp = sys-pid[i]; sys-pid[i] = sys-pid[j]; sys-pid[j] = temp;
temp = sys-index[i]; sys-index[i] = sys-index[j]; sys-index[j] = temp;
}
}
}

```

```

for(int i=0; i < wlen-1; i++)
    for(int j=i+1; j < wlen; j++)
        if(wlen-arm[i] > wlen-arm[j])
            int temp;

```

```

temp = wlen-arm[i]; wlen-arm[i] = wlen-arm[j]; wlen-arm[j] = temp;
temp = wlen-burst[i]; wlen-burst[i] = wlen-burst[j]; wlen-burst[j] = temp;
temp = wlen-pid[i]; wlen-pid[i] = wlen-pid[j]; wlen-pid[j] = temp;
temp = wlen-index[i]; wlen-index[i] = wlen-index[j]; wlen-index[j] = temp;
}
}
}

```

```

int time = 0;
printf("1a Gantt chart\n");

```

```

for(int i=0; i < sys-n; i++)
    if(time < sys-arm[i])
        time = sys-arm[i];

```

```

start[sys-index[i]] = time;
printf("P%d", sys-pid[i]);

```

```

time += sys-burst[i];

```

```

    ct[pgl-index[i]] = timeinter;
}

for(int i=0; i<cbor.n; i++){
    if(time < cbor.arr[i])
        time = cbor.arr[i];

    Start[cbor-index[i]] = time;
    printf("P%d 1", cbor-pid[i]);
    time += cbor-burst[i];
    ctct[cbor-index[i]] = time;
}

float total-tat = 0, total-wt = 0, total-rt = 0;
printf("\n\n P | P1 | A | B | C | D | E | F | G | H | I | J | K | L | M | N |");
for(int i=0; i<n; i++){
    tat[i] = ct[i] - arrival[i];
    wt[i] = tat[i] - burst[i];
    rt[i] = Start[i] - Arrival[i];

    total-tat += tat[i];
    total-wt += wt[i];
    total-rt = rt[i];

    printf("Y | D | Y | Y | Y | Y | Y | Y | Y | Y | Y | Y | Y | Y | Y | Y |", pid[i],
        arrival[i], burst[i], ct[i], tat[i], wt[i], rt[i]);
}

printf("\n\n Average Turnaround Time = %.2f", total-tat / n);
printf("\n\n Average Waiting Time = %.2f", total-wt / n);
printf("\n\n Average response Time = %.2f", total-rt / n);

return 0;
}

```

Output:

Enter number of Processes: 5

Process 1

Enter Arrival Time: 2

Enter Burst Time: 6

Enter type (1 = System, 2 = User): 1

Process 2

AT: 3

BT: 4

2

Process 3

AT: 5

BT: 6

2

Process 4

AT: 1

BT: 7

1

Process 5

AT: 4

BT: 9

2

Gantt chart

1 P4 | P1 | P2 | P5 | P3 |

PID	AT	BT	CT	TAT	WT	RT
1	2	6	14	12	6	6
2	3	4	18	15	11	11
3	5	6	53	28	22	22
4	1	7	8	7	0	0
5	4	9	27	23	14	14

Average TAT = 17.00

Average WT = 10.60

Average Response Time = 10.60

Write a C program to Simulate Real-time (RT) Scheduling algorithm:

a) Rate Monotonic

→

```
#include <stdio.h>
```

```
#define Max 10
```

```
typedef struct {  
    int id;  
    int bt;  
    int period;  
    int remaining;  
    int at next-arrival at;  
} process;
```

```
void SortByPeriod(process p[], int n) {  
    for (int i = 0; i < n - 1; i++) {  
        for (int j = i + 1; j < n; j++) {  
            if (p[i].period > p[j].period) {  
                process temp = p[i];
```

```
int gcd(int a, int b) {  
    while (b != 0) {  
        int t = b;  
        b = a % b;  
        a = t;  
    }  
    return a;  
}
```

```

int lcm (int a, int b) {
    return (a * b) / gcd (a, b);
}

void SortByPeriod (struct p [], int n) {
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = i + 1; j < n; j++) {
            if (p[i].period > p[j].period) {
                process temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
}

int main () {
    process p[MAX];
    int n;

    printf ("Enter number of processes : ");
    scanf ("%d", &n);

    float utilization = 0;

    for (int i = 0; i < n; i++) {
        p[i].id = i;
        printf ("\n Process %d : \n", i);
        printf ("Burst Time : ");
        printf ("%d", p[i].bt);

        printf ("Period : ");
        scanf ("%d", &p[i].periodod);
    }
}

```

utilization += (float) p[i].bt / p[i].period;

p[i].remaining = 0;

p[i].next-arrival = 0;

}

printf ("In CPU utilization = %.3f\n", utilization);

if (utilization > 1.0) {

printf ("Task Set is Not Schedulable (u > 1)\n");

return 0;

}

Part By Period (p, n);

int hyper = p[0].period;

for (int i = 1; i < n; i++) {

hyper = lcm (hyper, p[i].period);

}

int grant [1000];

for (int time = 0; time < hyper; time++) {

for (int i = 0; i < n; i++) {

if (time == p[i].next-arrival) {

p[i].remaining += p[i].bt;

p[i].next-arrival += p[i].period;

}

}

int sum = -1;

for (int i = 0; i < n; i++) {

if (p[i].remaining > 0) {

sum = i;

```

        break;
    }
}

if (run != -1) {
    gantt[time] = p[run].id;
    p[run].remaining--;
} else {
    gantt[time] = -1;
}
}

printf("\n Gantt chart (0 to x.d):\n", hyper);

for (int i = 0; i < hyper; i++)
    printf("-----");
printf("\n");

for (int i = 0; i < hyper; i++) {
    if (gantt[i] == -1)
        printf("1");
    else
        printf("P%d 1", gantt[i]);
}

printf("\n");

for (int i = 0; i < hyper; i++)
    printf("-----");
printf("\n");

for (int i = 0; i < hyper; i++)
    printf("%d - %d", i);
printf("\n");

return 0;
}

```

Output:

Enter no. of processes : 3

Process 0:

Burst Time : 3

Period : 6

Process 1:

BT : 2

Period : 4

Process 2:

BT : 1

Period = 3

CPU Utilization = 1.333

Task Set is NOT Schedulable (UT)

Enter no. of processes: 3

Process 0:

BT : 1

Period : 3

Process 1:

BT : 0

Period : 2

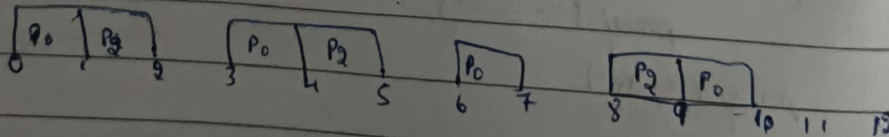
Process 2:

BT : 2

Period : 4

CPU = 0.833

Gantt chart (0 to 12)



b) Earliest - Deadline First Scheduling (EDF)

-2

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int main()
```

```
{
    int n;
```

```
    int arrival[MAX], burst[MAX], deadline[MAX], remaining[MAX];
```

```
    int time = 0, completed = 0;
```

```
    printf("Enter Number of tasks :");
```

```
    scanf("%d", &n);
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("Enter arrival, burst, deadline for T%d:", i+1);
```

```
        scanf("%d %d %d", &arrival[i], &burst[i], &deadline[i]);
```

```
        remaining[i] = burst[i];
```

```
    }
```

```
    printf("\n Time:");
```

```
    for (int i = 0; i < 30; i++)
```

```
        printf("%d ", i);
```

```
    printf("\n Task:");
```

```
    while (completed < n) {
```

```
        int idx = -1;
```

```
        int min_deadline = 9999;
```

```
        for (int i = 0; i < n; i++) {
```

```
            if (arrival[i] <= time && remaining[i] > 0) {
```

```
                if (deadline[i] < min_deadline) {
```

```
                    min_deadline = deadline[i];
```

```
                    idx = i;
```

```
            }
```

```
        }
```

```
    }
```

if (idx != -1)

printf("T%d", idx + 1);

remaining[idx]--;

if (remaining[idx] == 0)

Completed ++;

{ else

printf(" --");

}

time ++;

}

printf("\n");

return 0;

}

Output:

n = 3

T1: 0 3 7

T2: 0 2 4

T3: 0 2 5

$$v = \sum_{i=1}^n \frac{1}{T_i} = \frac{1}{1} = \frac{3}{1} + \frac{2}{4} + \frac{2}{5} =$$

Time: 0 1 2 3 4 5 6

Task: T2 T2 T3 T3 T1 T1 T1

(1)

Proportional Scheduling

-7 #include <stdio.h>

#define MAX 10

typedef struct {

int id;

int weight;

float share, allocated time;

} process;

int main() {

int n, total_weight = 0;

float total_cpu_time = 0;

process p[MAX];

printf("Enter the number of processes:");

scanf("%d", &n);

printf("Enter total CPU time:");

scanf("%d", &total_cpu_time);

for (int i = 0; i < n; i++) {

p[i].id = i+1;

printf("Enter weight of process %d", p[i].id);

scanf("%d", &p[i].weight);

total_weight += p[i].weight;

}

Date: Page:

```

for (int i=0; i<n; i++) {
    p[i].share = (float) p[i].weight / total_weight;
    p[i].allocated_time = p[i].share * total_cpu_time;
}
printf("In process id weight its share its allocated CPU time\n");
for (int j=0; j<n; j++) {
    printf("%d %d %d %d %d\n",
        p[j].id, p[j].weight, p[j].share,
        p[j].allocated_time);
}
return 0;
}

```

Output:

Enter the number of Processes : 4

Enter total CPU time available : 100

Enter P1 weight : 16

Enter P2 weight : 25

Enter P3 weight : 30

Enter P4 weight : 11

Process	Weight	Share	Allocated CPU Time
P1	16	0.19	19.05
P2	25	0.30	29.76
P3	30	0.38	38.10
P4	11	0.13	13.10

Practice - 1

The following program consists of 3 concurrent processes and 3 binary semaphores. The semaphores are initialized as $S_0 = 1, S_1 = 0, S_2 = 0$. Find out how many times Process P_0 will print "0". Assume the order of execution as P_0, P_1, P_2, P_0, P_1 .

Process P_0 :

```
while (true) {
    wait( $S_0$ );
    print("0");
    Signal( $S_1$ );
    Signal( $S_2$ );
}
```

Process P_1 :

```
wait( $S_1$ );
Signal( $S_0$ );
```

Process P_2 :

```
wait( $S_2$ );
Signal( $S_0$ );
```

-7

Initial	$S_0 = 1$	$S_1 = 0$	$S_2 = 0$	Remark
P_0	0	1	1	0
P_1	1	0	1	-
P_2	1	0	0	-
P_0	0	1	1	0
P_1	1	0	1	-

$\therefore P_0$ prints "0" 2 times.

Practice-2

wait → decrement
Signal → increment

Consider two processes A and B. Two ~~emp~~ Semaphore variables P and T are used to Synchronize. the process P is initialized to 0 and T is initialized to 1. Process are Scheduled in the following Order: ABABBABBA

What will be printed on the screen?

Process A

while (1) {

wait(P);

print(P);

Print(P);

Signal(T);

}

Process B

while (1) {

wait(T);

print(I);

print(I);

Signal(P);

}

→ Input	P=0	T=1	Remarks
A	0	1	Process A is blocked (wait)
B	1	0	II
A	0	1	IIPP
B	1	0	IIPP II
B	1	0	Process B is blocked (wait)
A	0	1	IIPP IIPP
A	0	0	IIPP Process A is blocked
B	1	0	IIPP IIPP II
A	0	1	IIPP IIPP IIPP

See
RA

Producer and Consumer

Buffer Size: 5

1) Produced: 0 at buffer [0] $\begin{matrix} 0 & 1 & 2 & 3 & 4 \\ [0, & -, & -, & -, & -] \end{matrix}$ 2) Consumed: 0 from buffer [0] $[-, & -, & -, & -, & -]$ 3) Produced: 1 at buffer [1] $[-, & 1, & -, & -, & -]$ 4) Consumed: 1 from buffer [1] $[-, & -, & -, & -, & -]$ 5) P: 2 at buffer [2] $[-, & -, & 2, & -, & -]$ 6) P: 3 at buffer [3] $[-, & -, & 2, & 3, & -]$ 7) C: 2 from buffer [2] $[-, & -, & -, & 3, & -]$ 8) P: 4 at b[4] $[-, & -, & -, & 3, & 4]$ 9) P: 5 at b[0] $[5, & -, & -, & 3, & 4]$ 10) C: 3 from b[3] $[5, & -, & -, & 4]$ 11) P: 6 at b[1] $[5, & 6, & -, & -, & 4]$ 12) P: 7 at b[2] $[5, & 6, & 7, & -, & 4]$ 13) C: 4 From b[4] $[5, & 6, & 7, & -, & -]$ 14) P: 8 at b[3] $[5, & 6, & 7, & 8, & -]$ 15) P: 9 at b[4] $[5, & 6, & 7, & 8, & 9]$ 16) C: 5 from b[0] $[-, & 6, & 7, & 8, & 9]$ 17) P: 10 at b[0] $[10, & 6, & 7, & 8, & 9]$ 18) C: 6 from b[1] $[10, & -, & 7, & 8, & 9]$ 19) P: 11 at b[1] $[10, & 11, & 7, & 8, & 9]$ 20) C: 7 from b[2] $[10, & 11, & -, & 8, & 9]$ 21) P: 12 at b[2] $[10, & 11, & 12, & 8, & 9]$ 22) C: 8 From b[3] $[10, & 11, & 12, & -, & 9]$ 23) P: 13 at b[3] $[10, & 11, & 12, & 13, & 9]$ 24) C: 9 From b[4] $[10, & 11, & 12, & 13, & -]$ 25) P: 14 at b[4] $[10, & 11, & 12, & 13, & 14]$

2) Producer and Consumer problem using Semaphores.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <pthread.h>
```

```
#include <semaphore.h>
```

```
#include <unistd.h>
```

```
int *buffer;
```

```
int size, n;
```

```
int in = 0, out = 0;
```

```
Sem_t empty, full;
```

```
pthread_mutex_t mutex;
```

```
void printBuffer () {
```

```
    printf ("Buffer (");
```

```
    for (int i=0; i<size; i++) {
```

```
        printf ("%d", buffer[i]);
```

```
        if (i != size-1) printf (" ");
```

```
    }
```

```
    printf ("]\n");
```

```
}
```

```
void *Producer (void *arg) {
```

```
    for (int i=1; i<=n; i++) {
```

```
        Sem_wait (&empty);
```

```
        pthread_mutex_lock (&mutex);
```

```
        buffer[in] = i;
```

```
        printf ("Produced : %d\n", i);
```

```
        printBuffer ();
```

```

in = (in + 1) % Size;
pthread_mutex_unlock(&mutex);
sem_post(&full);
Sleep(1);
}
return NULL;
}

```

```

void *Consumer(void *arg)
{

```

```

    int item;

```

```

    for (int i = 1; i <= n; i++) {

```

```

        sem_wait(&full);

```

```

        pthread_mutex_lock(&mutex);

```

```

        item = buffer[out];

```

```

        buffer[out] = 0;

```

```

        printf("Consumed: %d\n", item);

```

```

        printBuffer();

```

```

        out = (out + 1) % Size;

```

```

        pthread_mutex_unlock(&mutex);

```

```

        sem_post(&empty);

```

```

        Sleep(2);

```

```

    }

```

```

    return NULL;

```

```

}

```

```
int main()
```

```
{
```

```
pthread_t p, c;
```

```
printf("Enter buffer size:");
```

```
scanf("%d", &size);
```

```
printf("Enter number of items to produce:");
```

```
scanf("%d", &n);
```

```
buffer = (int *) malloc (size * sizeof (int));
```

```
for (int i=0; i<size; i++)
```

```
buffer[i] = 0;
```

```
Sem_init (&empty, 0, size);
```

```
Sem_init (&full, 0, 0);
```

```
pthread_mutex_init (&mutex, NULL);
```

```
pthread_create (&p, NULL, producer, NULL);
```

```
pthread_create (&c, NULL, consumer, NULL);
```

```
pthread_join (p, NULL);
```

```
pthread_join (c, NULL);
```

```
Sem_destroy (&empty);
```

```
Sem_destroy (&full);
```

```
pthread_mutex_destroy (&mutex);
```

```
free (buffer);
```

```
return 0;
```

```
}
```

b) Dining - Philosopher's problem

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
#define N 5
```

```
pthread_mutex_t forks[N];
```

```
void * philosopher(void * num){
```

```
    int id = *(int *) num;
```

```
    printf("philosopher %d is Thinking\n", id);
```

```
    sleep(1);
```

```
    if (id % 2 == 0){
```

```
        pthread_mutex_lock(&forks[(id+1)%N]);
```

```
        printf("Philosopher %d picked Right fork\n", id);
```

```
        pthread_mutex_lock(&forks[id]);
```

```
        printf("Philosopher %d picked Left fork\n", id);
```

```
    } else {
```

```
        pthread_mutex_lock(&forks[id]);
```

```
        printf("Philosopher %d picked Left fork\n", id);
```

```
        pthread_mutex_lock(&forks[(id+1)%N]);
```

```
        printf("Philosopher %d picked Right fork\n", id);
```

```
    }
```

```
    printf("Philosopher %d is Eating\n", id);
```

```
    sleep(2);
```

```
pthread_mutex_unlock(&forks[i]);
pthread_mutex_unlock(&forks[(i+1)%N]);
```

```
printf("Philosopher %d finished eating and exists\n", id);
```

```
pthread_exit(NULL);
```

```
}
```

```
int main()
```

```
{
    pthread_t philosophers[N];
```

```
    int ids[N];
```

```
    for (int i=0; i<N; i++) {
```

```
        pthread_mutex_init(&forks[i], NULL);
```

```
    }
```

```
    for (int i=0; i<N; i++) {
```

```
        ids[i] = i;
```

```
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
```

```
    }
```

```
    for (int i=0; i<N; i++) {
```

```
        pthread_join(philosophers[i], NULL);
```

```
    }
```

```
    printf("In All philosophers have eaten Once. Program ends.\n");
```

```
    return 0;
```

```
}
```

Output:

Philosopher 0 is Thinking

Philosopher 4 is Thinking

Philosopher 1 is Thinking

Philosopher 2 is Thinking

Philosopher 3 is Thinking

Philosopher 3 picked Left fork

Philosopher 4 picked Right fork

Philosopher 1 picked Left fork

Philosopher 1 picked Right fork

Philosopher 1 is Eating

Philosopher 3 picked Right fork

Philosopher 3 is Eating

Philosopher 1 finished eating and exits

Philosopher 4 picked Left fork

Philosopher 4 is eating

Philosopher 2 picked Right fork

Philosopher 2 picked Left fork

Philosopher 2 is eating

Philosopher 0 picked Right fork

Philosopher 3 finished eating & exits

Philosopher 2 finished eating & exits

Philosopher 4 finished eating & exits

Philosopher 0 picked Left fork

Philosopher 0 is eating

Philosopher 0 finished eating & exits

Write a C program to simulate

Baker's algorithm for the purpose of deadlock avoidance.

a)

→

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int main()
```

```
{
    int n, m;
```

```
    printf("Enter number of processes :");
```

```
    scanf("%d", &n);
```

```
    printf("Enter number of resources :");
```

```
    scanf("%d", &m);
```

```
    int alloc[MAX][MAX], max[n][MAX], need[n][MAX];
```

```
    int avail[MAX], finish[MAX] = {0};
```

```
    int safeflag[MAX];
```

```
    printf("\n Enter Allocation Matrix : \n");
```

```
    for (int i = 0; i < n; i++)
```

```
    {
        for (int j = 0; j < m; j++)
```

```
        {
            scanf("%d", &alloc[i][j]);
```

```
        }
```

```
    }
```

```
    printf("\n Enter Maximum Matrix : \n");
```

```
    for (int i = 0; i < n; i++)
```

```
    {
        for (int j = 0; j < m; j++)
```

```
        {
            scanf("%d", &max[i][j]);
```

```
        }
```

```
    }
```

```

printf("\n Resources Enter Available Resources:\n");
for (int i=0; i<m; i++)
    scanf("%d", &avail[i]);
}

```

```

printf("\n Need Matrix :\n");
for (int i=0; i<n; i++)
    for (int j=0; j<m; j++)
        need[i][j] = max[i][j] - avail[i][j];
        printf("%d", need[i][j]);
}

```

```

printf("\n");
}

```

```

int count=0;

```

```

while (count < n)

```

```

    int p=0;

```

```

    for (int i=0; i<n; i++)
    {

```

```

        if (p[i][0]==0)
        {

```

```

            int i;

```

```

            for (j=0; j<m; j++)
            {

```

```

                if (need[i][j] > avail[j])
                {

```

```

                    break;
                }

```

```

            }
            if (j==m)
            {

```

```

                for (int k=0; k<m; k++)
                {

```

```

                    avail[k] += need[i][k];
                }

```

```

            }

```

SafeSeq[Count+1] = i;

finish[i] = 1;

found = 1;

}

}

}

if (found == 0) {

printf("No System is not in a Safe State (Deadlock exists)");

return 0;

}

}

printf("No System is in a Safe State. In Safe Sequence :");

for (i = 0; i < n; i++)

printf("%d", SafeSeq[i]);

}

printf("\n");

return 0;

}

Output

Enter number of process: 5

Enter number of resources: 3

Enter Allocation matrix	Enter Maximum Matrix	Enter Available Resource
-------------------------	----------------------	--------------------------

010	753	333
-----	-----	-----

300	322	
-----	-----	--

302	902	
-----	-----	--

211	222	
-----	-----	--

002	433	
-----	-----	--

Need Matrix:

743

123

600

011

431

System is in a Safe State

Safe Sequence: P1 P3 P4 P0 P2.

// Resource Request Algorithm

int process;

int request[MAX];

printf("\n Enter process number making request: ");

scanf("%d", &process);

printf("\n Enter request vector: ");

for (int i = 0; i < M; i++)

scanf("%d", &request[i]);

for (int i = 0; i < M; i++)

if (request[i] > need[process][i])

printf("\n Error: Request exceeds need\n");

return 0;

}

}

for (int i = 0; i < M; i++)

if (request[i] > avail[i])

printf("\n Resources not available, process must wait\n");

return 0;

}

```

for (int i = 0; i < n; i++) {
    avail[i] -= request[i];
    alloc[process][i] += request[i];
    need[process][i] -= request[i];
}

for (int i = 0; i < n; i++)
    finish[i] = 0;

count = 0;

while (count < n) {
    int found = 0;

    for (int i = 0; i < n; i++) {
        if (finish[i] == 0) {
            int j;

            for (j = 0; j < m; j++) {
                if (need[i][j] > avail[j])
                    break;
            }

            if (j == m) {
                for (int k = 0; k < m; k++)
                    avail[k] += alloc[i][k];
                request[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }

    if (found == 0) {
        printf("No Process can be granted (unsafe state)\n");
        return 0;
    }
}

```

```
printf("No Request can be granted. System remains Safe.\n");  
for (int i=0; i<n;i++)  
    printf("P%d", S[i]);  
printf("\n");  
return 0;
```

Output:

Enter process number making request: 1

Enter request vector:

1 0 2

Request can be granted. System remains Safe.

Safe Sequence: P0 P1 P2 P3 P4

Deadlock Detection Algorithm

```
#include <stdio.h>
#define MAX 10

int main() {
    int n, m;
    printf("Enter Number of processes:");
    scanf("%d", &n);
    printf("Enter number of resources:");
    scanf("%d", &m);

    int alloc[MAX][MAX], request[MAX][MAX];
    int avail[MAX], finish[MAX], seq[MAX];

    printf("Initial Allocation Matrix: \n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            scanf("%d", &request[i][j]);

    printf("Initial Request Matrix: \n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            scanf("%d", &request[i][j]);

    printf("Initial Available Resources: \n");
    for (int i = 0; i < m; i++)
        scanf("%d", &avail[i]);

    for (int i = 0; i < n; i++)
        finish[i] = 0;
```

```

int Count = 0;
while (Count < N) {
    int pword = 0;

    for (int i = 0; i < N; i++) {
        if (first[i] == 0) {
            int j;
            for (j = 0; j < M; j++) {
                if (request[i][j] > avail[j])
                    break;
            }
            if (j == M) {
                for (int k = 0; k < M; k++)
                    avail[k] += alloc[i][k];

                safeSeq[Count++] = i;
                first[i] = 1;
                pword = 1;
            }
        }
    }

    if (pword == 0) {
        printf("\n System is Not in a Safe State\n");
        return 0;
    }
}

printf("\n System is in Safe State\n State Sequence : ");
for (int i = 0; i < N; i++) {
    printf("P%d", safeSeq[i]);
    printf("\n");
    return 0;
}

```

Output:

Enter Number of Processes: 5

Enter Number of Resources: 3

Enter Allocation Matrix

	Request	Available
010	000	000
200	202	
303	000	
211	100	
002	002	

System is in Safe State.

Safe Sequence: P0 P3 P4 P1

#

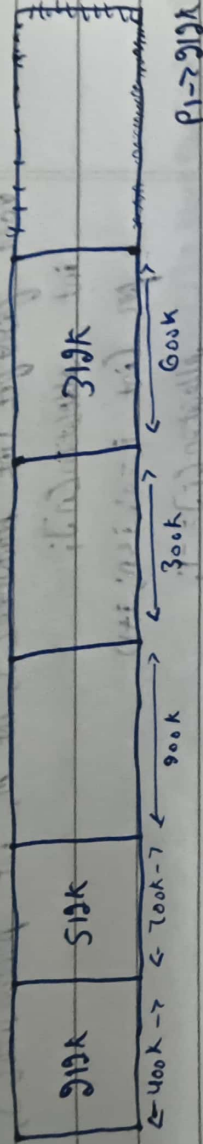
Allocation	Request	Available
010	000	000
200	202	
303	001	
211	100	
002	002	

System is not in Safe State, and Process P1, P3, P4 and P2 are Caused Deadlock.

✓

Given a free memory partition 400K, 700K, 200K, 300K & 600K how could each of first fit, best fit & worst-fit algorithm work to place process of 212K, 512K, 312K, 596K, 512K.

First fit



P1 → 212K

~~512K cannot be allocated with memory.~~

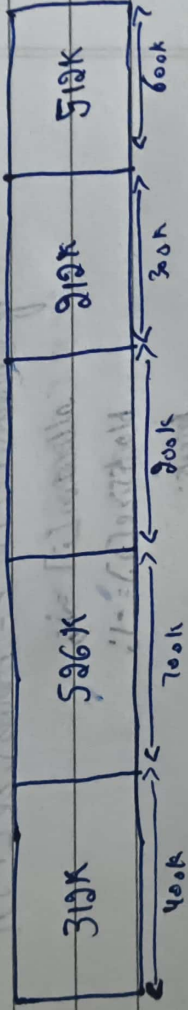
P2 → 512K

P3 → 312K

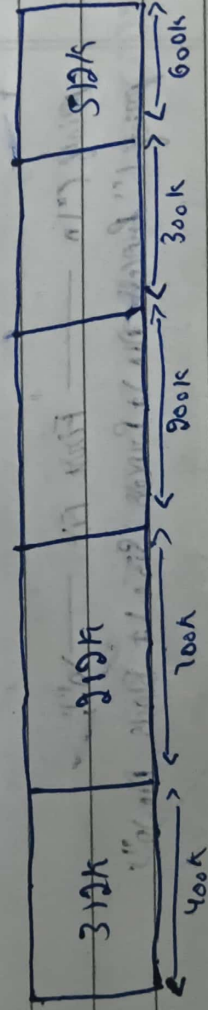
P4 → 596K

596K cannot be allocated with memory.

Best fit



Worst fit



596K cannot be allocated with memory.

Write a program to simulate the following Contiguous Memory allocation techniques.

- a) Worst-fit b) Best-fit c) First-fit

→

```
#include <stdio.h>
```

```
void firstfit (int blockSize[], int m, int processSize[], int n) {
    int allocation[n];
```

```
    for (int i = 0; i < n; i++)
```

```
        allocation[i] = -1;
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < m; j++) {
```

```
            if (blockSize[j] >= processSize[i]) {
```

```
                allocation[i] = j;
```

```
                blockSize[j] = -1;
```

```
                break;
```

```
            }
```

```
        }
```

```
    }
```

```
    printf("\n ----- First Fit -----");
```

```
    printf("\n Process No. & Process Size & Block No.");
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("%d & %d & %d", i+1, processSize[i],
```

```
            if (allocation[i] != -1)
```

```
                printf("%d", allocation[i]+1);
```

```
            else
```

```
                printf("Not Allocated");
```

```
        }
```

```

void bestFit (int blockSize[], int n, int processSize[], int m) {
    int allocation[n];
    for (int i=0; i<n; i++)
        allocation[i] = -1;
    for (int i=0; i<n; i++)
        int bestIdx = -1;
        for (int j=0; j<m; j++) {
            if (blockSize[j] == processSize[i]) {
                if (bestIdx == -1) || blockSize[j] < blockSize[bestIdx]) {
                    bestIdx = j;
                }
            }
        }
        if (bestIdx != -1) {
            allocation[i] = bestIdx;
            blockSize[bestIdx] = -1;
        }
    }

    printf("\n --- Best Fit ---\n");
    printf("Process No. \t process size \t Block No. \n");
    for (int i=0; i<n; i++) {
        printf("Process No. \t %d \t KB \t %d", i+1, processSize[i]);
        if (allocation[i] != -1)
            printf(" \t %d", allocation[i]+1);
        else
            printf(" \t Not Allocated");
    }
}

```

```

void Graphs (int blockSize[], int n, int processSize[], int n) {
    int allocation[n];
    for (int i = 0; i < n; i++)
        allocation[i] = -1;
    for (int i = 0; i < n; i++) {
        int curBlock = -1;
        for (int j = 0; j < n; j++) {
            if (blockSize[j] == processSize[i]) {
                if (curBlock == -1 || blockSize[j] > blockSize[curBlock]) {
                    curBlock = j;
                }
            }
        }
        if (curBlock != -1) {
            allocation[i] = curBlock;
            blockSize[curBlock] = -1;
        }
        printf("%d\n", allocation[i] == -1 ? "Not allocated" : "Allocated", allocation[i]);
    }
    for (int i = 0; i < n; i++) {
        printf("%d\n", allocation[i] == -1 ? "Not allocated" : "Allocated", allocation[i]);
    }
}

```

```

int main() {
    int n, n1;
    printf("Enter number of memory blocks: ");
    scanf("%d", &n1);
    int block1[n], block2[n], block3[n];
    printf("Enter size of each memory block: \n");
    for (int i=0; i<n; i++) {
        scanf("%d", &block1[i]);
        block2[i] = block1[i];
        block3[i] = block1[i];
    }
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int processSize[n];
    printf("Enter size of each process: \n");
    for (int i=0; i<n; i++) {
        scanf("%d", &processSize[i]);
    }
    printf("block1, n, processSize, n1);
    printf("block2, n1, processSize, n1);
    printf("block3, n1, processSize, n1);
    return 0;
}

```

Output:

Enter number of memory blocks: 5

Enter size of each memory block:

400, 700, 900, 500, 600.

Enter number of processes: 4

Enter size of each process:

212, 312, 512, 526

First Fit

Process No.	Process Size	Block No.
1	912 KB	1
2	512 KB	2
3	312 KB	5
4	526 KB	Not Allowed.

Next Fit

Process No.	Process Size	Block No.
1	912 KB	4
2	512 KB	5
3	312 KB	1
4	526 KB	2

Worst Fit

Process No.	Process Size	Block No.
1	912 KB	2
2	512 KB	5
3	312 KB	1
4	526 KB	Not Allowed.